tsvc/s212 · s212.c · bare_llm · claude-haiku-4-5-20251001 · BROKEN

−0 / +18 lines of 176 · 2 parallel region(s) added · 0 of 0 candidate(s) kept · RESTRUCTURING (2 code line(s) added, 0 removed) · restructuring level 0

BEFORE

```
132  }
133
134  static real_t kernel_s212(void)
135  {
136      for (int nl = 0; nl < R; nl++) {
137          for (int i = 0; i < LEN_1D-1; i++) {
138              a[i] *= c[i];
139              b[i] += a[i + 1] * d[i];
140          }
141          pb_mix(nl);
142      }
143      return (real_t)0;
144  }
145
```

AFTER

```
132  }
133
134  static real_t kernel_s212(void)
135  {
136      for (int nl = 0; nl < R; nl++) {
137          /* Phase 1: Compute a[i] *= c[i] for all i in parallel.
138           * Each iteration writes a unique a[i] and reads the same a[i] and c[i
139           * No inter-iteration dependencies; parallelizable.
140           * Variables: i is private (induction var), a and c are shared arrays.
141           */
142          #pragma omp parallel for
143          for (int i = 0; i < LEN_1D-1; i++) {
144              a[i] *= c[i];
145          }
146
147          /* Phase 2: Compute b[i] += a[i+1] * d[i] for all i in parallel.
148           * The barrier between phases ensures all a[i] values are updated befo
149           * any phase 2 iteration reads a[i+1], eliminating the original loop-c
150           * Each iteration writes a unique b[i] and reads a[i+1] (now finalized
151           * No inter-iteration dependencies; parallelizable.
152           * Variables: i is private (induction var), a, b, d are shared arrays.
153           */
154          #pragma omp parallel for
155          for (int i = 0; i < LEN_1D-1; i++) {
156              b[i] += a[i + 1] * d[i];
157          }
158
159          pb_mix(nl);
160      }
161      return (real_t)0;
162  }
163
```

file

red = removed green = added yellow = parallel region (#pragma omp) right strip = the whole file: green where changed, yellow where a parallel region now is